

**Universidad de Guadalajara
(Centro Universitario de los
Valles)**



Desarrollo de Base de Datos para Web

Profesor:

Adolfo Castillo Chavarin

Alumno: Aceves Lara

Jonathan Alexis

22/09/2026

Juegos

Agregar

Nombre	Descripción	Precio	Editar	Eliminar
Max Steel	muñeco de accion	400.00	Editar	Eliminar

Agregar juego

Nombre:

Max Steel

Descripción:

muñeco de accion

Precio:

1299

Guardar

Volver

Juegos

Agregar

Nombre	Descripción	Precio	Editar	Eliminar
Max Steel	muñeco de accion	400.00	Editar	Eliminar
Max Steel	muñeco de accion	1299.00	Editar	Eliminar

1. Introducción

Descripción del Proyecto

En esta práctica desarrollé una aplicación web para la administración de un catálogo de videojuegos siguiendo el patrón **Modelo-Vista-Controlador (MVC)**. El sistema utiliza **Python** con **Flask** en el *backend*, **PyMySQL** para la conexión con **MySQL/MariaDB**, y plantillas HTML con **Jinja2** estilizadas mediante **Bulma CSS**.

Objetivos

- **General:** Construir un sistema web funcional con operaciones CRUD (*Crear, Leer, Actualizar y Eliminar*).
- **Específicos:**
 - Configurar la conexión entre Python y MySQL mediante pymysql.
 - Encapsular la lógica SQL en un controlador independiente (`controlador_juegos.py`).
 - Mapear rutas en Flask (`main.py`) para procesar solicitudes GET y POST.
 - Diseñar plantillas responsivas utilizando herencia HTML con Jinja2 y Bulma CSS.

2. Desarrollo

Pasos de Implementación

1. **Base de Datos:** Inicié el servidor local en XAMPP y creé la base de datos juegos con la tabla juegos (id, nombre, descripcion, precio).
 2. **Conexión (bd.py):** Definí la función `obtener_conexion()` para establecer el enlace con el servidor MySQL local.
 3. **Controlador (controlador_juegos.py):** Programé las funciones para ejecutar consultas SQL parametrizadas (%s), previniendo inyecciones SQL.
 4. **Rutas (main.py):** Mapeé las rutas en Flask para renderizar vistas y procesar datos enviados por formularios con redirección automática a /juegos.
 5. **Vistas (templates/):** Diseñé `maestra.html` como plantilla base y extendí su estructura hacia `juegos.html`, `agregar_juego.html` y `editar_juego.html`.
- **Captura 1: Estructura de la Base de Datos:** Tabla juegos en MySQL con sus columnas.
 - **Captura 2: Pantalla Principal (/juegos):** Tabla con el catálogo de juegos cargado y los botones de acción.
 - **Captura 3: Formulario de Registro / Edición:** Vista con los campos Nombre, Descripción y Precio.
 - **Captura 4: Confirmación del Cambio:** Resultado en la tabla tras modificar o eliminar un registro.

3. Lecciones Aprendidas

- **Separación de Responsabilidades (MVC):** Mantener las consultas SQL fuera de las rutas de Flask facilita la lectura y mantenimiento del proyecto.
- **Manejo de Conexiones:** Comprendí que las transacciones en base de datos requieren `commit()` para guardar cambios y `close()` para liberar recursos.
- **Modularidad con Jinja2:** La herencia de plantillas (`{% extends %}`) evita duplicar código HTML en las distintas vistas.

4. Conclusiones

Se logró implementar con éxito un mini-sistema web completo, integrado y funcional sobre Flask y MySQL. La práctica permitió afianzar el flujo de datos entre el navegador, el servidor *backend* y la base de datos relacional, sentando las bases para desarrollar aplicaciones web más complejas en el futuro.

